



**UNIVERSIDAD DE
COSTA RICA**
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 9: Consola Logística

"ExpresoFast" - Procedimientos

Almacenados, Paginación Relacional y

Vistas HTML5 Paginadas

Metadatos

- **Tiempo Estimado:** 6 horas (trabajo autónomo individual)

Herramientas Requeridas:

- Java 21 (o versión local instalada) con Maven.
- Spring Boot 3.x (Spring Data JPA, Spring Security, JWT).
- H2 Database o SQL Server Developer Edition.
- Navegador Web moderno (Google Chrome / Firefox) con DevTools.

- Editor de código (Visual Studio Code / IntelliJ IDEA).
- Git y Cuenta activa de GitHub.

Metas de Aprendizaje:

Diseñar e implementar procedimientos almacenados (*Stored Procedures*) nativos en el motor de base de datos para la generación de reportes y filtrado logístico.

Mapear e invocar procedimientos almacenados desde la capa de datos en Spring Data JPA utilizando la anotación `@Procedure` y `EntityManager`.

Implementar paginación relacional a nivel de base de datos (`Pageable`, `PageRequest`, `Page<T>`) para prevenir saturación de memoria RAM al consultar grandes volúmenes de registros.

Construir un controlador RESTful que retorne payloads JSON paginados estandarizados con metadatos de navegación y ordenamiento.

Desarrollar una interfaz cliente web responsiva en HTML5, CSS3 y Vanilla JavaScript (Fetch API) que consuma la API paginada y renderice controles interactivos de paginación (`Primera`, `Anterior`, `Página X de Y`, `Siguiente`, `Última`).

Introducción y Caso de Estudio (Quinta Parte)

En el **Laboratorio 8**, su equipo desplegó la **Consola Web de Operación Logística** para la compañía **ExpresoFast**, conectando la interfaz cliente semántica HTML5 y CSS3 con la API RESTful protegida por tokens JWT.

Dado el crecimiento del volumen diario de paquetes y guías de envío procesadas por ExpresoFast, la Gerencia de Tecnología exige optimizar el rendimiento del sistema back-end y front-end. Cargar listas masivas de envíos en memoria sin paginación degrada la JVM y bloquea la experiencia de usuario.

Para esta **Quinta Parte**, usted debe extender autónomamente la plataforma **ExpresoFast** construyendo dos módulos de optimización:

Módulo de Procedimientos Almacenados (*Stored Procedures*): Rutinas precompiladas en la base de datos para consultar y resumir envíos por estado sin sobrecargar el ORM.

Módulo de Paginación Relacional Web: Consultas paginadas físicamente a nivel de SQL (`Pageable`, `Page<T>`) y una tabla interactiva HTML5 con botones de navegación paginada.

Requerimientos Técnicos del Laboratorio

1. Stored Procedures en Base de Datos (`schema.sql` y `data.sql`)

Usted debe crear y ejecutar en su base de datos relacional (H2 o SQL Server) el esquema de la tabla `Envio` y los siguientes procedimientos almacenados nativos:

- **[] Procedimiento `SP_OBTENER_ENVIOS_POR_ESTADO`:**
- Parámetro de entrada: `pEstado` (VARCHAR/String).

Salida: Conjunto de resultados (*Result Set*) con los registros de envíos que coincidan con dicho estado, ordenados descendientemente por `fecha_creacion`.

[] Procedimiento `SP_RESUMEN_METRICAS_ENVIOS` (Opcional para puntos extra / Reto):

Calcula el conteo total de envíos y la suma acumulada de montos de flete agrupados por estado.

[] Datos de Prueba (`data.sql`):

- Inserte al menos 15 registros de semillas con diversos estados (`PENDIENTE`, `EN_TRANSITO`, `ENTREGADO`, `CANCELADO`) para verificar la paginación.

2. Capa de Datos y Mapeo JPA (`EnvioRepository.java` y `EnvioDTO.java`)

En la capa de persistencia de su aplicación Spring Boot:

[] **DTO EnvioDTO**: Defina un record o clase DTO para transferir únicamente los campos requeridos (`id`, `codigoRastreo`, `destinatario`, `direccionDestino`, `montoFlete`, `estado`, `fechaCreacion`).

[] **Mapeo @Procedure**:

En `EnvioRepository`, declare el método mapeado a `SP_OBTENER_ENVIOS_POR_ESTADO` utilizando `@Procedure` y `@Param("pEstado")`.

[] **Consultas Paginadas con Pageable**:

- Declare métodos de consulta paginada que reciban la interfaz `Pageable` de Spring Data (ejemplo: `findByEstado(String estado, Pageable pageable)` y búsquedas por término o palabra clave).

3. Capa de Negocio y Controlador RESTful (`EnvioService.java` y `EnvioController.java`)

En la capa back-end:

- [] **Servicio de Negocio (EnvioService)**:
- Implemente el método `listarPaginado(int page, int size, String sortBy, String dir, String busqueda, String estado)` que construya el objeto `PageRequest.of(page, size, Sort.by(...))` y retorne un objeto `Page<EnvioDTO>`.

Implemente el método `listarViaStoredProcedure(String estado)` que invoque el procedimiento almacenado relacional.

[] **Controlador RESTful (EnvioController)**:

- Exponga el endpoint `GET /api/v1/envios` aceptando parámetros opcionales de consulta: `page` (default 0), `size` (default 5), `sortBy`, `direction`, `busqueda` y `estado`.
- Exponga el endpoint `GET /api/v1/envios/procedimiento/{estado}` para retornar la lista de envíos generada por el Stored Procedure.

4. Consola Web Cliente Paginada HTML5, CSS3 y JavaScript

(`dashboard_paginado.html`)

En el front-end cliente, integre una vista paginada en su consola logística:

- [] **Estructura HTML5 Semántica:**
- Un formulario/barra de filtros con caja de texto de búsqueda, selector desplegable de Stored Procedure y selector de tamaño de página (5, 10, 20 ítems por página).

Tabla semántica `<table>` para desplegar las guías de envío (Rastreo, Destinatario, Dirección, Flete, Estado).

[] **Barra de Navegación de Paginación:**

- Botones interactivos: `<< Primera`, `< Anterior`, `Siguiente >`, `Última >>`.
- Indicador de estado accesible: `"Página X de Y (Total: Z envíos)"`.

Deshabilitación automática de botones (`disabled`) cuando el cliente se ubique en la primera página (`data.first`) o última página (`data.last`).

[] **Consumo Asíncrono (Fetch API / `async-await`):**

- Realizar peticiones `fetch()` hacia `/api/v1/envios` pasando `page` y `size`.
- Renderizar dinámicamente las filas del cuerpo `<tbody>` y actualizar los metadatos del paginador sin recargar la página web.

Sección de Depuración de Errores Comunes

Error 1: Desajuste de Índice Base 0 en Spring Data (`IndexOutOfBoundsException`)

- [] Inspeccione la consola DevTools (F12) pestaña **Network** al cambiar de página en la UI.
- [] Recuerde que Spring Data interpreta `page = 0` como la primera página. Presente al usuario `data.number + 1` en pantalla pero envíe `currentPage` (base 0) a la API REST.

Error 2: Advertencia de Paginación en Memoria (HHH000104)

- [] Verifique sus consultas JPQL en `EnvioRepository.java`.
- [] No combine la cláusula `JOIN FETCH` sobre colecciones `@OneToMany` con el parámetro `Pageable`. Utilice `@EntityGraph(attributePaths = {...})` para evitar el desbordamiento de memoria RAM.

Entregables y Modalidad de Entrega

- **Modalidad:** Trabajo Autónomo Individual.

Repositorio en GitHub:

- Suba el proyecto estructurado en dos carpetas principales: `/backend` (proyecto Spring Boot) y `/frontend` (archivos HTML, CSS y JS).
- Incluya un archivo `README.md` con instrucciones para ejecutar la API REST y desplegar el cliente en un servidor local.

Entrega en Mediación Virtual:

- Suba un documento PDF o archivo comprimido que contenga:
 - Enlace al repositorio público de GitHub.
 - Capturas de pantalla de la Consola Paginada funcionando en el navegador web con botones de navegación activos.
 - Capturas de la ejecución de los Stored Procedures desde la interfaz web.

Rúbrica de Evaluación (Total: 100 Puntos)

Criterio de Evaluación	Puntuación	Descripción
Stored Procedures en Base de Datos y Mapeo JPA	25 pts	Definición correcta del procedimiento almacenado nativo en SQL y mapeo exitoso en Spring Data JPA mediante @Procedure.
Paginación Relacional Back-End (Pageable)	25 pts	Implementación de consultas paginadas físicas en EnvioRepository y EnvioService con respuestas Page<EnvioDT0>.
Controlador RESTful Paginado	15 pts	Exposición de endpoints GET /api/v1/envios con soporte de parámetros de paginación (page, size, sortBy, direction).
Interfaz Web Cliente HTML5 Paginada	25 pts	Desarrollo autónomo de la tabla HTML5 con controles interactivos de paginación (Primera, Anterior, Siguiente, Última) y selector de tamaño de página.
Manejo de Errores y Depuración	10 pts	Corrección de desajustes de índice base 0/1 y ausencia de advertencias de paginación en memoria (HHH000104).